ML Report: Sanskrit Ciphers — Computational Puzzle-Solving for Ancient Buddhist Textual History

Omar El Aref, Dylan Garner, Muhammad Umar Khan,
Aswin Kuganesan, Yousef Shahin
{elareo,garnerd4,khanm101,kuganea,shahiny1}@mcmaster.ca

1 Introduction

The textual history of Indian Buddhism is fragmented across thousands of manuscript folios preserved only in partial, damaged, or scattered forms. Scholars traditionally reconstruct these texts through paleographic study, manual transcription, and visual comparison of physical fragments—a slow, labor-intensive, and error-prone process that does not scale to the tens of thousands of surviving fragments.

Sanskrit Ciphers is a machine-learning-powered computational platform that automates the detection, classification, and matching of ancient Buddhist manuscript fragments. The system applies computer vision and deep learning to extract structural metadata from fragment images—including segmentation masks, text line counts, script type, edge morphology, scale calibration, and the presence of circular annotations—enabling researchers to search, filter, and reconstruct manuscripts through an interactive digital workspace.

This report describes the ML models developed for the project, the datasets used to train and evaluate them, the features and architectures employed, the results achieved, and our plans for further improvement.

2 Related Work

The computational analysis of manuscript fragments draws on techniques from document image analysis, object detection, and image classification.

? present a framework for analyzing ancient manuscript fragments using image features and geometric matching, demonstrating that computer vision can assist scholars in identifying relationships between fragments. The YOLO family of detectors (**??**) has become a standard for real-time object detection and instance segmentation; we adopt YOLOv8 for both fragment segmentation and text line detection. Transfer learning from ImageNet-pretrained backbones (**?**) is well established for small-dataset regimes; we leverage MobileNetV2 (**?**) for script type classification. **?** show that data augmentation and architectural choices such as batch normalization significantly improve CNN performance on visual document tasks—practices we apply throughout our pipeline. EasyOCR (**?**) provides a ready-to-use optical character recognition engine that we integrate for ruler text detection in our scale calibration processor.

No prior system, to our knowledge, combines all of these capabilities—segmentation, line detection, script classification, edge analysis, circle detection, and scale estimation—into a single pipeline for Sanskrit manuscript fragments.

3 Dataset

Our dataset consists of high-resolution photographs of Sanskrit Buddhist manuscript fragments provided by Dr. Shayne Clarke (McMaster University, Department of Religious Studies). The images vary in resolution (typically 1500–4000 px on the long edge), depict fragments on heterogeneous backgrounds, and include a physical ruler at the bottom of each image for scale reference.

3.1 Labeling and Annotation

- **Segmentation:** Polygon masks around fragments were annotated manually and converted to YOLO segmentation format.
- **Line detection:** Bounding boxes around individual text lines were annotated in YOLO detection format with a single class (`line`).
- **Line count classification:** A CSV file maps each fragment image filename to its manually counted number of text lines (0–15).
- **Script type classification:** A CSV file maps each fragment to one of five script types: *Early South Turkestan Brahmi*,

North Turkestan Brahmi, *North Turkestan Brahmi Type a*, *South Turkestan Brahmi*, and *Turkestan Gupta Type*.

- **Circle detection:** Segmentation masks for circular annotation marks were annotated in YOLO segmentation format.

3.2 Preprocessing

All models that accept fixed-size input (classification, script type) resize images to 224×224 or 256×256 pixels and normalize with ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$). YOLO-based models (segmentation, line detection, circle detection) operate on the native resolution with internal resizing handled by Ultralytics. For scale detection, the bottom 15% of each image is extracted and passed through EasyOCR for ruler text localization.

3.3 Data Splits

Stratified splits were used for all classification tasks:

- **Line count CNN:** 75% train / 15% validation / 10% test (stratified by line count class).
- **Script type classifier:** 80% train / 10% validation / 10% test (stratified by script type), with a `WeightedRandomSampler` applied to the training set to address class imbalance.
- **YOLO models:** Standard YOLO train/val splits defined in `data.yaml` configuration files.

4 Features

Our pipeline does not rely on hand-crafted feature engineering for the deep learning models; instead, features are learned end-to-end within the neural networks:

- **CNN line count model:** Learns hierarchical convolutional features through four blocks of increasing depth ($64 \to 128 \to 256 \to 512$ channels), ending with adaptive average pooling to produce a 512-dimensional feature vector.
- **MobileNetV2 script type classifier:** Leverages pretrained ImageNet features via transfer learning. Early backbone layers are frozen; only the final 20 parameters and a custom classification head are fine-tuned, producing a 1280-dimensional feature vector before classification.
- **YOLOv8 models:** Use the built-in YOLOv8 feature pyramid network for multi-scale feature extraction.
- **Edge detection:** Uses classical computer vision features (contour extraction, convex hull segmentation, line fitting via least-squares) rather than learned features. Straight-edge candidates are identified by measuring orthogonal residuals against fitted lines.
- **Scale detection:** Combines OCR-detected text features (ruler unit labels) with signal processing features (intensity peak detection via Gaussian smoothing and `scipy.signal.find_peaks`) to identify ruler tick marks.

Data augmentation is applied during training for the CNN and MobileNetV2 models: random horizontal flips, color jitter (brightness, contrast, saturation, hue), random rotation (± 10), random affine transformations, random perspective distortion, sharpness adjustment, and auto-contrast.

5 Implementation

The ML pipeline is implemented in Python using PyTorch (**?**) and Ultralytics YOLOv8 (**?**). All models are orchestrated through a modular processor architecture: each model is encapsulated as a `BaseProcessor` subclass with standardized `process()`, `should_process()`, and `cleanup()` interfaces. A YAML configuration file controls which processors are enabled, their model paths, confidence thresholds, and versioning.

5.1 Model Architectures

Segmentation (YOLOv8-seg). A YOLOv8 segmentation model predicts pixel-level masks for manuscript fragments. The processor extracts contours from the predicted mask, generates a transparent PNG with the background removed, and stores contour coordinates as JSON in the database. Confidence threshold: 0.25; IoU threshold: 0.45.

Line Count Classification (Custom CNN). A four-block CNN (`LineCountCNN`) classifies fragments into 16 classes (0–15 lines). Each block consists of two 3×3 convolution layers with batch normalization, ReLU activation, max pooling, and spatial dropout (0.1–0.2). The classifier

head uses three fully connected layers ($512 \to 256 \to 128 \to 16$) with batch normalization and dropout (0.4/0.3/0.2). Trained with cross-entropy loss, Adam optimizer ($\text{lr} = 5 \times 10^{-4}$, weight decay 10^{-5}), and ReduceLROnPlateau scheduler (patience 5, factor 0.5). Early stopping with patience 15.

Script Type Classification (MobileNetV2). An `EfficientScriptTypeClassifier` uses a pretrained MobileNetV2 backbone with frozen early layers and a custom head: Dropout(0.3) $\to$ Linear(1280, 256) $\to$ BatchNorm $\to$ ReLU $\to$ Dropout(0.2) $\to$ Linear(256, 5). Trained with cross-entropy loss, Adam optimizer ($\text{lr} = 10^{-3}$, weight decay 10^{-4}), and ReduceLROnPlateau scheduler. A `WeightedRandomSampler` addresses class imbalance. Gradient clipping (max_norm $= 1.0$) is applied.

Line Detection (YOLOv8-detect). A YOLOv8 object detection model identifies individual text lines in fragments, outputting bounding boxes sorted top-to-bottom. Confidence threshold: 0.25; IoU threshold: 0.45.

Circle Detection (YOLOv8-seg). A YOLOv8 segmentation model detects circular annotation marks in fragments, outputting a binary has/does-not-have-circle flag. Confidence threshold: 0.25.

Edge Detection (Classical CV). An `EdgeExtractor` classifies each of the four sides (top, bottom, left, right) as either a straight border or a tear. Two methods are implemented: (1) bounding-box-proximity runs with line-fit residual thresholds, and (2) convex-hull-based straight segment extraction with angle and coverage filters. The pipeline uses the hull-segments method by default.

Scale Detection (EasyOCR + Signal Processing). The `ScaleDetectionProcessor` detects ruler tick marks using a hybrid approach: EasyOCR localizes "cm" or "mm" text in the bottom image region; adaptive peak detection identifies tick positions; and the median inter-tick gap yields a pixels-per-unit ratio.

5.2 Training Infrastructure

All models were trained on personal machines with CUDA-enabled GPUs. Training times ranged from minutes (MobileNetV2 with 20 epochs) to several hours (YOLOv8 segmentation). PyTorch reproducibility was enforced with fixed seeds (`seed=42`).

Model / Task	Recall	F1 Score
Circle detection	98.5%	0.992
Segmentation	99.7%	0.999
Line detection	94.8%	0.855
Classification (CNN)	87.7%	0.802
Scale detection	81.3%	0.880
Script type	82.6%	0.792
Edge detection	77.2%	0.698

Table 1: ML model evaluation results (Recall and F1 Score).

6 Results and Evaluation

Evaluation was conducted as part of the project's Verification and Validation (V&V) process. Table **??** summarizes the recall and F1 scores across all seven ML tasks.

6.1 Analysis

Strong performers. Segmentation (F1 = 0.999) and circle detection (F1 = 0.992) achieve near-perfect scores. The segmentation model benefits from clear visual contrast between fragments and backgrounds; circle detection is a relatively constrained binary task with distinctive visual features.

Moderate performers. Line detection (F1 = 0.855) and scale detection (F1 = 0.880) perform well but show room for improvement. Line detection errors tend to occur on heavily damaged fragments where text lines are partially occluded. Scale detection struggles with images where the ruler is faded, partially cropped, or uses non-standard labeling.

Challenging tasks. Script type classification (F1 = 0.792) and edge detection (F1 = 0.698) are the most difficult. Script type classification contends with subtle visual differences between related Brahmi script families and limited training data for some classes. Edge detection relies on geometric heuristics that can misclassify irregular but intentional borders as tears, particularly on fragments with unusual shapes.

6.2 Baselines

For line count classification, a majority-class baseline (always predicting the most common line

count) achieves approximately 15% accuracy, confirming that the CNN model (87.7% recall) provides substantial improvement. For script type classification, a random baseline yields approximately 20% accuracy (1 in 5 classes), compared to the model's 82.6% recall.

6.3 Limitations

- **Dataset size:** Some script type classes have limited representation, leading to class imbalance despite weighted sampling.
- **Overfitting risk:** The custom CNN for line counting is relatively deep for the dataset size; dropout and augmentation mitigate this but do not eliminate the risk.
- **Edge detection heuristics:** The classical CV approach lacks the generalization capability of a learned model and is sensitive to threshold parameters.
- **Scale detection robustness:** Images without visible rulers or with damaged rulers yield no scale data.

7 Feedback and Plans

Feedback from our TA highlighted the need to strengthen our evaluation methodology and provide deeper analysis of failure cases. Specifically, the TA suggested that we include more granular per-class metrics for the classification models and provide confusion matrices to better understand where misclassifications occur. The TA also recommended that we discuss the trade-offs between our classical computer vision approaches (edge detection, scale detection) and potential learned alternatives.

Based on this feedback and our own analysis, we plan the following improvements for the remainder of the project:

- **Edge detection:** Explore training a lightweight learned model to replace or augment the heuristic-based edge classifier, which currently has the lowest F1 (0.698).
- **Script type classification:** Collect additional training data for underrepresented script classes and experiment with larger pretrained backbones (e.g., EfficientNet or ResNet-50) while monitoring for overfitting.
- **Scale detection:** Improve robustness to faded or partially cropped rulers by incorporating template matching as a fallback when OCR-based detection fails.
- **Per-class evaluation:** Generate and analyze confusion matrices for all classification tasks and report per-class precision, recall, and F1.
- **Cross-validation:** Implement k-fold cross-validation for the smaller classification datasets to obtain more reliable performance estimates.

Team Contributions

- **Dylan Garner:** Led the ML pipeline architecture and integration. Implemented the modular processor framework, database schema, and the line count classification CNN. Contributed to the segmentation model and auto-scaling detection.
- **Yousef Shahin:** Developed the YOLOv8 line detection model, including data annotation, training pipeline, and evaluation scripts. Contributed to dataset organization.
- **Aswin Kuganesan:** Developed the script type classification system (MobileNetV2 model, training pipeline, data preparation, evaluation module) and integrated it into the ML pipeline.
- **Omar El Aref:** Developed the YOLOv8 segmentation model for fragment extraction, implemented edge detection algorithms (bounding-box and convex-hull methods), contributed unit tests for the ML pipeline, and generated evaluation figures.
- **Muhammad Umar Khan:** Contributed to edge detection algorithms, implemented scale detection processor, circle detection integration, front-end integration, and pipeline configuration.